

Padrão Decorator em Java

Objetivo

Modelar a arquitetura de um sistema simples utilizando o padrão de projeto **Decorator**, conforme visto em aula.

Contexto

Um sistema de catálogo de veículos exibe informações na tela.

A exibição básica de um veículo pode ser enriquecida com:

- informações técnicas do modelo
- logo da marca

A solução deve permitir adicionar essas funcionalidades **dinamicamente**, sem modificar a classe principal.

Estrutura esperada

O sistema deve seguir o padrão **Decorator**, com:

Interface

- ComponenteGraficoVeiculo

Componente concreto

- VisaoVeiculo

Decorator abstrato

- Decorador

Decoradores concretos

- DecoradorModelo
- DecoradorMarca
-

Tarefa

Elaborar o **diagrama de classes UML completo** do sistema.

Restrições

-  Não incluir classes de teste
-  Não incluir código Java (pode preparar o código que será apresentado diretamente na aula de laboratório)
-  Representar apenas a arquitetura

Elementos obrigatórios

Interface

ComponenteGraficoVeiculo

- `exibir() : void`

Classe

VisaoVeiculo

- implementa `ComponenteGraficoVeiculo`
- `exibir() : void`

Classe abstrata

Decorator

- implementa `ComponenteGraficoVeiculo`
- atributo:
 - `componente : ComponenteGraficoVeiculo`
- construtor com parâmetro `componente`
- `exibir() : void`

Classes concretas

DecoratorModelo

- herda de `Decorator`
- `exibirInformacoesTecnicas() : void`
- `exibir() : void`

DecoratorMarca

- herda de `Decorator`
- `exibirLogoMarca() : void`
- `exibir() : void`

O método `exibir()` **não retorna valor**.

Ele deve realizar a saída diretamente utilizando `System.out.println()`.

Exemplo simples

```
public void exibir() {  
    System.out.println("Exibição básica do veículo");  
}
```

Relações obrigatórias

- VisaoVeiculo → implementa interface
- Decorador → implementa interface
- DecoradorModelo e DecoradorMarca → herdam de Decorador
- Decorador → possui referência para ComponenteGraficoVeiculo

 Representar claramente:

- herança
- implementação de interface
- associação (composição)

Observação

A implementação será realizada **no laboratório**, com:

- programa de teste fornecido
- código desenvolvido a partir deste diagrama

Cenário de referência

Considere a seguinte construção:

```
ComponenteGraficoVeiculo v = new VisaoVeiculo();  
v = new DecoradorModelo(v);  
v = new DecoradorMarca(v);
```

Ao chamar: `v.exibir();`
a saída deve ser:

Exibição básica do veículo
Informações técnicas do modelo
Logo da marca

Interpretação

- VisaoVeiculo → exibição básica
- DecoradorModelo → adiciona informações técnicas
- DecoradorMarca → adiciona o logo

 Cada decorador:

- delega a chamada (`super.exibir()`)
- acrescenta seu próprio comportamento

Objetivo

Seu diagrama deve permitir exatamente esse tipo de encadeamento dinâmico de funcionalidades.

Questões conceituais

1. Qual problema o padrão Decorator resolve neste exemplo?
2. Por que Decorador implementa ComponenteGraficoVeiculo?
3. Qual é a função do atributo componente em Decorador?
4. Por que DecoradorModelo e DecoradorMarca herdam de Decorador?
5. O que aconteceria se a ordem dos decoradores fosse invertida?
6. Por que essa solução é mais flexível do que criar subclasses para todas as combinações possíveis?
7. Identifique no diagrama: componente, componente concreto, decorador abstrato e decoradores concretos.
8. Explique por que os decoradores devem delegar a chamada ao objeto decorado antes de adicionar seu comportamento.